

AI World Atlas - Nucleo Navegable - Diseno

Diseño — Núcleo Navegable de AI World Atlas

Fecha: 2026-09-25 **Estado:** Especificación para revisión **Alcance:** primera entrega funcional — “Núcleo Navegable” **Repositorio destino:** <https://github.com/LeonardoPS1/ia-world-atlas>

1. Contexto

1.1 Estado actual del repositorio

El repositorio local es una materialización parcial de la V4 descrita en la conversación pública compartida:

- Frontend Vite + Mapbox GL JS en JavaScript, monolítico y actualmente roto por un error de sintaxis en `apps/web/src/main.js:14`.
- API Express + `pg` en un único archivo, con escritura anónima mediante `POST /api/projects` en `apps/api/src/server.js:13`.
- PostgreSQL 16 + PostGIS 3.4 mediante Docker Compose.
- Esquema base `locations`, `projects`, `events` y `relations` en `apps/api/sql/001_schema.sql`.
- Seed inicial de 9 ubicaciones y 5 proyectos en `apps/api/sql/002_seed.sql`, con fuentes genéricas y métricas semánticamente ambiguas. El seed objetivo de esta entrega lo reemplaza por 11 ubicaciones que cubren los seis niveles de jerarquía.
- Sin tests, lint, typecheck, migraciones, lockfiles, CI ni repositorio Git inicializado.

1.2 Hallazgos del diagnóstico

Severidad	Hallazgo	Ubicación
P0	El frontend no parsea; la aplicación queda en blanco	<code>apps/web/src/main.js:14</code>
P0	<code>POST /api/projects</code> anónimo con CORS abierto por defecto	<code>apps/api/src/server.js:6,13</code>
P0	Datos externos interpolados con <code>innerHTML</code> ; riesgo XSS almacenado	<code>apps/web/src/main.js:10,12</code>
P1	Listener de errores de Mapbox registrado después de <code>style.load</code> ; fallback no cubre fallos reales de token o red	<code>apps/web/src/main.js:14</code>
P1	Filtros aplicados sólo a la capa <code>points</code> ; clusters sin mismo filtro	<code>apps/web/src/main.js:11,14</code>
P1	Vocabularios de <code>type</code> , <code>status</code> y <code>evidence</code> sin <code>CHECK</code> ; jerarquía sin <code>CITY</code> / <code>LOCAL_AREA</code>	<code>apps/api/sql/001_schema.sql:3,7</code>
P1	Fuentes sin <code>source_type</code> , <code>confidence</code> , <code>snippet</code> ni historial de estado	<code>apps/api/sql/001_schema.sql</code>
P1	Detalle inaccesible en tablet y móvil; paneles fijos reduce el mapa	<code>apps/web/src/style.css:1</code>
P1	Carga silenciosamente truncada a 500 proyectos; índice GIN no utilizado por <code>ILIKE</code>	<code>apps/api/src/server.js:9</code> , <code>001_schema.sql:22</code>
P2	Scripts SQL sólo se ejecutan en la primera creación del volumen	<code>docker-compose.yml:11-15</code>
P2	Errores 500 exponen mensajes internos de PostgreSQL	<code>apps/api/src/server.js</code>
P2	<code>last_verified_at = now()</code> en el seed sugiere verificación no realizada	<code>apps/api/sql/002_seed.sql</code>

1.3 Contraste con la conversación compartida

Coinciden el stack, la proyección Globe, la ausencia de rotación automática, el fallback, la paleta, el PostGIS y la intención de separar UI de datos.

No coinciden ni deben conservarse: marcadores circulares sólidos, layout de dashboard con paneles permanentes, controles `WORLD/COUNTRY/CITY` sin comportamiento, escritura anónima, métricas de “verificado” sin auditoría y fuentes genéricas de dominios institucionales.

Los artefactos de código V1–V4 del chat están redactados en el enlace público. La conversación sirve como requisito de producto, no como evidencia técnica.

2. Objetivos

1. Permitir navegar desde Viña del Mar hasta el planeta y volver, con jerarquía territorial explícita.
2. Mostrar únicamente registros de IA con estado, evidencia, fecha, fuente y fragmento verificable.
3. Convertir el mapa en el elemento principal, ocupando entre 70% y 80% del espacio visual en desktop.
4. Ofrecer búsqueda, filtros combinables, clusters coherentes y detalle contextual.
5. Incluir una timeline básica basada exclusivamente en fechas documentadas.
6. Garantizar fallback funcional y diagnóstico cuando Mapbox no esté disponible.
7. Funcionar en desktop, tablet y móvil con navegación por teclado y reduced motion.

8. Dejar una arquitectura modular, testeada y preparada para Dokploy.

3. No objetivos de esta entrega

- Ingesta automática de noticias, deduplicación y entity resolution.
 - Admin/backoffice, autenticación de usuarios y My Atlas.
 - Grafo visual de relaciones, comparador y notificaciones.
 - AI Pulse avanzado, AI Stack, AI Invisible, Future Radar, Story Mode y Possibility Lab.
 - Modo claro, internacionalización, cuentas, app nativa y vector tiles.
 - Despliegue productivo en VPS; sólo preparación verificable.
-

4. Decisiones cerradas

ID	Decisión	Motivo
D1	Primera entrega = Núcleo Navegable	Evitar una demo amplia difícil de convertir en producto
D2	Vite + TypeScript modular, sin React	Mantener control del mapa y bajo acoplamiento
D3	Seed curado y auditado; sin news ingestion	Evitar datos inventados y dependencia de medios
D4	Runtime local + Docker Compose	Reproducibilidad y preparación VPS
D5	Mapbox GL JS Globe + fallback propio	Requisito del SRD y resiliencia ante token/WebGL/red
D6	Dark editorial; mapa 70–80%	Identidad “instrumento, no dashboard”
D7	Cinco registros auditados	Suficiente para validar local → global sin simular densidad
D8	Arquitectura incremental modular	Conservar Vite/Mapbox/Express/PostGIS y endurecer por capas
D9	Filtres y búsqueda en la API	Una única fuente de verdad para mapa, clusters, fallback y listas
D10	Migraciones SQL versionadas	Los scripts de inicialización no actualizan volúmenes existentes
D11	Workspace <code>packages/contracts</code>	Compartir esquemas Zod y tipos entre API y web
D12	Sin ruta de escritura pública en V1	Eliminar la escritura anónima antes de exponer la API

5. Auditoría de fuentes del seed

Reglas:

- `VERIFIED` exige revisión auditable con fuente primaria comprobada; no se asigna automáticamente por estar en un sitio oficial.
- `REPORTED` representa una afirmación documentada por la organización o un medio identificable.
- `ANNOUNCED` representa un plan, programa o compromiso futuro aún no materializado.
- `confidence` es `HIGH`, `MEDIUM` o `LOW` y califica la fuente, no la trascendencia del proyecto.
- No se incluyen cifras sin fuente primaria accesible ni claims como “58 centros” o “20 soluciones”.

ID propuesto	Título	Ubicación	Tipo	Estado	Evidencia	Fuente primaria	Claim permitido
pucv-fondecyt-fuzzy	An Adaptive Fuzzy Control System for Metaheuristic Configuration	Valparaíso	RES EAR CH	RESE ARCH	REPORT ED	https://www.ingenieria.pucv.cl/desarrollan-proyecto-de-inteligencia-artificial-con-algoritmos-capables-de-aprender-de-su-propio-desempeno/	Proyecto Fondecyt Regular que aplica control difuso adaptativo a algoritmos metaheurísticos; uno de los 31 proyectos adjudicados a académicos PUCV en 2026.
chile-national-ai-policy	Política Nacional de Inteligencia Artificial	Chile	POL ICY	ACTI VE	REPORT ED	https://www.minciencia.gob.cl/areas/inteligencia-artificial/politica-nacional-de-inteligencia-artificial/	Política vigente desde 2021, con plan de acción y actualización del Eje 3; la versión actualizada reporta 177 acciones, 100 comprometidas para 2026 y coordinación de 14 ministerios.
indiaai-mission	IndiaAI Mission	India	GOV ERN MEN T	ACTI VE	REPORT ED	https://indiaai.gov.in/article/indiaai-innovation-challenge-2026 y https://indiaai.gov.in/article/union-minister-dharmendra-pradhan-launches-three-ai-centres-of-excellence	IndiaAI, IBD de Digital India Corporation bajo MeitY, implementa la mission nacional; se documentan tres Centres of Excellence lanzados en octubre de 2024 con ₹990 crore.
punggol-digital-district	Punggol Digital District Open Digital Platform	Singapur	INF RAS TRU CTU RE	ACTI VE	REPORT ED	https://www.smartnation.gov.sg/initiatives/smart-city-solutions/	Plataforma que integra sistemas de smart city para monitoreo en tiempo real y optimización de recursos en el distrito. No se afirma "Physical AI" ni robótica específica sin fuente adicional.
eu-ai-factories	European AI Factories	Unión Europea	INF RAS	DEPL OYIN	ANNOUN CED	https://digi	19 AI Factories y 13 antenas en preparación, al menos 9

ID propuesto	Título	Ubicación	Tipo	Estado	Evidencia	Fuente primaria	Claim permitido
S			TRU CTU RE	G		strategy.ec.europa.eu/en/policies/ai-factories	supercomputadores optimizados, llamada hasta 7 AI Gigafactories y hasta €10bn de apoyo público; la inversión privada esperada se etiqueta como expectativa, no hecho consumado.

Cada proyecto tendrá al menos una fuente primaria en el seed, un evento documental con la fecha de publicación y ninguna relación si las fuentes no documentan una relación explícita.

6. Arquitectura

6.1 Estructura del monorepo

```

ai-world-atlas/
├─ apps/
│  └─ api/
│     ├── migrations/
│     ├── seeds/
│     ├── src/
│     │  ├── config/
│     │  ├── db/
│     │  ├── errors/
│     │  ├── middleware/
│     │  ├── repositories/
│     │  ├── routes/
│     │  ├── schemas/
│     │  ├── services/
│     │  ├── testing/
│     │  └─ server.ts
│     └─ Dockerfile
├─ web/
│  ├── src/
│  │  ├── api/
│  │  ├── data/
│  │  ├── map/
│  │  ├── state/
│  │  ├── ui/
│  │  ├── styles/
│  │  └─ main.ts
│  └─ Dockerfile
├─ packages/
│  └─ contracts/
├─ .github/workflows/ci.yml
├─ docker-compose.yml
├─ docker-compose.prod.yml
├─ package.json
├─ tsconfig.base.json
└─ README.md

```

6.2 Responsabilidades

Unidad	Responsabilidad	Responsabilidad excluida
<code>packages/contracts</code>	Vocabularios cerrados, esquemas Zod, tipos y shape de errores	Acceso a datos o DOM
<code>apps/api</code>	REST, validación, repositorios, PostGIS, migraciones y seed	Lógica visual
<code>apps/web</code>	Estado, Mapbox, fallback, UI, accesibilidad y render seguro	Consultas SQL o secretos
Docker Compose	PostgreSQL/PostGIS reproducible y healthcheck	Secretos de producción hardcodeados

6.3 Flujo de arranque

1. Validar variables de entorno con Zod y fallar rápido si faltan.
2. Consultar `GET /api/health`.
3. Obtener `GET /api/locations` y `GET /api/projects?pageSize=200`.
4. Normalizar respuestas con los esquemas compartidos.
5. Derivar `visibleProjects`, breadcrumb, clusters, timeline y stats desde el mismo estado.
6. Renderizar shell editorial; montar Mapbox sólo cuando token, WebGL2 y red son utilizables.
7. Ante cualquier fallo de mapa, montar fallback de datos sin perder selección ni filtros.

7. Contratos compartidos

`packages/contracts` exportará:

- `PROJECT_TYPES` con 16 valores: `PROJECT`, `NEWS`, `LAUNCH`, `COMPANY`, `GOVERNMENT`, `UNIVERSITY`, `RESEARCH`, `INFRASTRUCTURE`, `ROBOTICS`, `POLICY`, `INVESTMENT`, `EDUCATION`, `APPLICATION`, `IMPACT`, `SIGNAL`, `POSSIBILITY`.
- `PROJECT_STATUSES` con 12 valores: `IDEA`, `RESEARCH`, `ANNOUNCED`, `FUNDED`, `PILOT`, `BUILDING`, `DEPLOYING`, `ACTIVE`, `SCALING`, `COMPLETED`, `PAUSED`, `CANCELLED`.
- `EVIDENCE_LEVELS`: `VERIFIED`, `REPORTED`, `ANNOUNCED`, `ANALYSIS`, `SIGNAL`, `POSSIBILITY`.
- `LOCATION_LEVELS`: `WORLD`, `CONTINENT`, `COUNTRY`, `REGION`, `CITY`, `LOCAL_AREA`.
- `SOURCE_TYPES`: `GOVERNMENT`, `UNIVERSITY`, `ORGANIZATION`, `COMPANY`, `PAPER`, `MEDIA`, `OTHER`.
- `CONFIDENCE_LEVELS`: `HIGH`, `MEDIUM`, `LOW`.
- `RELATION_TYPES`: `PARTNERSHIP`, `FUNDING`, `RESEARCH`, `INFRASTRUCTURE`, `GOVERNMENT`, `SUPPLIER`, `UNIVERSITY`, `DEPLOYMENT`, `LOCATION`, `POLICY`, `TECHNOLOGY`, `INVESTMENT`.
- Schemas `ProjectSummary`, `ProjectDetail`, `Location`, `Event`, `Relation`, `Source`, `StatusHistoryEntry`, `StatsResponse`, `ApiError`.

La web parseará respuestas con estos esquemas; cualquier deriva de contrato falla durante desarrollo en lugar de producir una pantalla inconsistente.

8. Modelo de datos

8.1 locations

- `id text primary key`
- `name text not null`
- `level text not null check (level in ('WORLD', 'CONTINENT', 'COUNTRY', 'REGION', 'CITY', 'LOCAL_AREA'))`
- `parent_id text references locations(id)`
- `country_code char(2)`
- `geography geography(Point,4326) not null`
- `metadata jsonb not null default '{}'`
- `created_at, updated_at timestampz`

La validación de jerarquía padre-hijo se hace en servicio; la FK protege integridad referencial.

8.2 projects

- `id text primary key`
- `name text not null`
- `type text not null` con `CHECK` de 16 valores
- `status text not null` con `CHECK` de 12 valores
- `evidence text not null` con `CHECK` de 6 valores
- `sector text`
- `summary text not null`
- `impact text`
- `year integer check (year between 1900 and 2100)`
- `location_id text not null references locations(id)`
- `geometry geography(Point,4326)`
- `actors text[] not null default '{}'`
- `tags text[] not null default '{}'`
- `published_at timestampz`
- `last_verified_at timestampz not null`
- `created_at, updated_at timestampz`

Si `geometry` es nulo, la API usa el centroide de `location_id`; no se inventan coordenadas.

8.3 sources

- `id text primary key`
- `name text not null`
- `url text not null`
- `source_type text not null` con `CHECK`
- `publication_date date`

- `last_verified_at timestamptz not null`
- `confidence text not null check (confidence in ('HIGH','MEDIUM','LOW'))`
- `created_at, updated_at timestamptz`

La API valida que `url` use `https:` en producción y `http:` sólo en desarrollo local.

8.4 `project_sources`

- `project_id text references projects(id) on delete cascade`
- `source_id text references sources(id)`
- `snippet text not null`
- `is_primary boolean not null default false`
- `primary key (project_id, source_id)`

8.5 `status_history`

- `id bigserial primary key`
- `project_id text not null references projects(id) on delete cascade`
- `from_status text` con el mismo `check` cerrado de estados; `null` sólo en la primera transición
- `to_status text not null`
- `changed_at timestamptz not null`
- `note text`
- `source_id text references sources(id)`

El historial es append-only; la API V1 no expone mutaciones.

8.6 `impact_records`

- `id bigserial primary key`
- `project_id text not null references projects(id) on delete cascade`
- `category text not null check (category in ('ECONOMIC', 'SOCIAL', 'EDUCATIONAL', 'HEALTH', 'ENVIRONMENTAL', 'INFRASTRUCTURE', 'REGULATORY', 'OTHER'))`
- `description text not null`
- `source_id text references sources(id)`

8.7 `events` y `relations`

- `events`: `project_id`, `title`, `occurred_at date`, `kind`, `description`, `source_id`.
- `relations`: `from_project`, `to_project`, `type` con 12 valores, `description`, `source_id`, `UNIQUE(from_project,to_project,type)`.

El seed no crea relaciones si la fuente no las documenta; el frontend muestra un estado vacío honesto.

8.8 Índices

- GIST en `locations.geography` y `projects.geometry`.

- B-tree en `projects(type)`, `projects(status)`, `projects(evidence)`, `projects(location_id)`, `projects(published_at)`, `projects(last_verified_at)`.
- GIN sobre una expresión inmutable, registrada como `projects_search_idx`:
`to_tsvector('spanish'::regconfig, coalesce(name,'') || ' ' || coalesce(summary,'') || ' ' || coalesce(array_to_string(tags, ' '), ''))`. El `::regconfig` fijo es obligatorio: PostgreSQL exige que la configuración sea constante para indexar.
- La búsqueda usa el mismo predicado que el índice, con `plainto_tsquery('spanish', q)` y Ranking por `ts_rank`; el `ILIKE` queda sólo como fallback para tokens de menos de 3 caracteres.
- B-tree en `sources(source_type)`, `events(occurred_at)`, `status_history(project_id, changed_at)`.

9. Migraciones y seed

9.1 Runner

Un runner TypeScript basado en `pg`:

1. Crea `schema_migrations(name text primary key, applied_at timestamptz)`.
2. Lee `apps/api/migrations/*.sql` en orden lexicográfico.
3. Aplica cada archivo dentro de una transacción.
4. Registra el archivo sólo después de una transacción exitosa.
5. No ejecuta migraciones ya registradas.

Scripts raíz:

- `npm run db:migrate`
- `npm run db:seed`
- `npm run db:setup` = migrate + seed
- `npm run db:reset` = documentado para destruir el volumen y reconstruir desde cero.

9.2 Archivos

- `migrations/001_core.sql`: esquema final, idempotente, con vocabularios cerrados, fuentes, historial, impacto e índices. En una base vacía produce el estado objetivo completo.
- `migrations/002_upgrade_v4.sql`: upgrade desde el esquema legacy V4 (`001_schema.sql` + `002_seed.sql`). En una base creada por `001_core.sql` es un no-op, porque todas las sentencias son `ADD COLUMN IF NOT EXISTS` / `CREATE TABLE IF NOT EXISTS` / `DO $$ ... $$` con guardas de existencia. En una base legacy añade lo que falta, migra filas válidas y descarta sólo registros que violan los vocabularios cerrados, reportando el conteo en el log del runner.
- `seeds/001_core_seed.sql`: 11 locations, 5 projects, al menos 5 sources primarias, al menos 5 events documentales y 0 relations inventadas. Todas las inserciones son idempotentes (`ON CONFLICT DO UPDATE` sobre claves naturales) para que `db:seed` sea reejecutable.

Las 11 locations del seed cubren los seis niveles del vocabulario:

id	level	notes
<code>world</code>	WORLD	raíz
<code>south-america</code>	CONTINENT	padre de <code>chile</code>
<code>chile</code>	COUNTRY	
<code>valparaiso-region</code>	REGION	
<code>valparaiso</code>	CITY	centro de la región
<code>vina-del-mar</code>	CITY	entry point de la experiencia
<code>santiago</code>	CITY	segunda ciudad del seed
<code>pucv-campus</code>	LOCAL_AREA	requiere un punto exacto; se usa la dirección oficial declarada por la fuente PUCV, no una coordenada inventada
<code>india</code>	COUNTRY	
<code>singapore</code>	COUNTRY	
<code>eu</code>	COUNTRY	agregación del bloque de factories; modelado como COUNTRY por limitación de un solo nivel geográfico, documentado como limitación conocida

El endpoint de desarrollo espera el healthcheck de PostGIS antes de ejecutar `db:setup` y luego inicia API y web.

10. API v1

Todas las respuestas usan JSON. Los errores usan:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request",
    "details": [],
    "requestId": "..."
  }
}
```

Método	Ruta	Query / uso	Respuesta
GET	/api/health	—	{ status, api, database, version, time }
GET	/api/locations	parentId, level, page, pageSize	{ data: Location[], page, pageSize, total, totalPages }
GET	/api/projects	q, type, status, evidence, sector, locationId, yearFrom, yearTo, page, pageSize, sort	{ data: ProjectSummary[], page, pageSize, total, totalPages }
GET	/api/projects/:id	—	{ data: ProjectDetail }
GET	/api/events	projectId, yearFrom, yearTo	{ data: Event[], count }
GET	/api/relations	projectId	{ data: Relation[], count }
GET	/api/stats	—	{ totals, byEvidence, byType, byStatus } — totales globales de todo el corpus, sin filtros

Reglas:

- `page` inicia en 1; `pageSize` por defecto 50 y máximo 200. Ambos aplican a `/api/locations` y a `/api/projects`.
- `sort` acepta `publishedAt` y `name`.
- `type`, `status` y `evidence` aceptan repetición (`?type=POLICY&type=RESEARCH`) y valores separados por coma (`?type=POLICY,RESEARCH`), con semántica OR dentro del mismo parámetro y AND entre parámetros distintos.
- `q` usa `websearch_to_tsquery('spanish', ...)` con fallback `ILIKE` para nombres parciales.
- `locationId` incluye descendientes mediante CTE recursiva.
- `GET /api/projects/:id` incluye `location`, `sources`, `events`, `relations`, `statusHistory` e `impactRecords`.
- No existe `POST /api/projects`; una escritura anónima devuelve 404/405.
- Los 500 devuelven `INTERNAL_ERROR` sin stack ni mensaje PostgreSQL.
- `CORS_ORIGIN` es una allowlist separada por comas; `*` sólo si se define explícitamente en desarrollo.
- `helmet` se activa; body JSON máximo 1 MB; requests se identifican con `requestId`.

11. Frontend

11.1 Sistema visual

Tokens CSS en `src/styles/tokens.css`:

```

--color-obsidian: #08090C;
--color-ink: #11141A;
--color-warm-white: #E9E6DF;
--color-electric-blue: #4B8CFF;
--color-ultraviolet: #8567FF;
--color-mint: #70D6B2;
--color-amber: #E5A84B;
--color-ice: #70C8E8;
--color-coral: #E47C71;
--color-lime: #A9D66F;
--color-lavender: #B59AFF;
--duration-fast: 160ms;
--duration-base: 280ms;
--duration-slow: 420ms;
--ease-atlas: cubic-bezier(0.22, 0.61, 0.36, 1);

```

- Sans geométrica del sistema para UI; serif local sólo para wordmark y títulos editoriales.
- Iconos SVG inline lineales; no emojis ni librerías de iconos genéricas.
- Bordes finos, superficies ink, sin glassmorphism, neon ni cards repetitivas.
- Transiciones sin bounce; reduced motion elimina pulsos, órbitas y timings largos.

11.2 Layout

Desktop (`>= 1100px`):

- Header superior: wordmark, breadcrumb, búsqueda, estado API/mapa.
- Rail izquierdo colapsable: escala territorial, filtros y leyenda.
- Mapa a pantalla completa detrás, con 70–80% del área visible.
- Drawer derecho sólo aparece al seleccionar.
- Strip inferior: timeline, evidencia y stats.

Tablet (`700–1099px`): rail plegable y drawer en panel lateral de 360 px.

Mobile (`< 700px`): mapa protagonista, rail como sheet y detalle como bottom sheet; no se oculta información sin alternativa.

11.3 Marcadores jerárquicos

Un generador puro `renderMarkerSvg(kind, color, size)` produce imágenes SDF para capas `symbol`:

Alcance	Forma	Animación
<code>LOCAL_AREA</code>	anillo abierto + núcleo translúcido	pulso leve o ninguno con reduced motion
<code>REGION</code>	doble anillo radar	respiración mínima
<code>COUNTRY</code>	hexágono abierto	rotación muy lenta o respiración
<code>CONTINENT</code>	arco orbital	recorrido de perímetro
<code>WORLD</code>	nodo/constelación	puntos conectados

- Prohibido `circle` sólido como pin de proyecto.
- Clusters: anillo abierto con contador, zoom al expansion zoom.
- `feature-state` `selected` añade anillo eléctrico; no cambia la cámara automáticamente.

- Colores por categoría: research mint, government/policy amber, infrastructure ice, robotics/application coral, signal lime, possibility lavender, default electric blue.

11.4 Fallback

- Sin token: diagnóstico `VITE_MAPBOX_TOKEN no configurado`.
- Token inválido o error de estilo: diagnóstico con status y timestamp.
- WebGL2 ausente: diagnóstico de capacidad.
- Fallback renderiza datos reales como puntos sobre graticula equirectangular explícitamente etiquetada `Coordinate view`, más lista navegable; no simula continentes ni límites.
- La selección, búsqueda, filtros, timeline y detalle siguen operativos.

11.5 Estado y selectores

Store único:

```
data: locationsById, projectsById, events, relations, stats
query: q, type[], status[], evidence[], sector, locationId, year
view: selectedProjectId, selectedLocationId, activeLevel, timelineYear, mapMode, railOpen,
detailOpen
connection: health, mapDiagnostic, loading, error, lastFetchedAt
```

Selectores puros:

- `visibleProjects`: respuesta ya filtrada por la API, ordenada por el store.
- `projectById`
- `locationAncestors`
- `timelineRange`
- `clusterSource`
- `statsForStrip` combina dos fuentes, cada una etiquetada en la UI para que nunca se confundan:
 - `totals` (globales, sin filtros), servidos por `GET /api/stats` y cacheados en el store bajo `data.stats`.
 - `matchingCount` (resultado de la consulta actual), tomado del campo `total` de `GET /api/projects` y almacenado junto a la última consulta. El strip muestra `X registros coinciden` (derivado de `matchingCount`) y, en un secondary line, `Y en total` (de `totals`). La acceptance 7 se cumple porque mapa, clusters, lista y `matchingCount` provienen de la misma respuesta de la API; `totals` es deliberadamente global y se rotula como tal.
- `documentCountByYear`

La web no vuelve a filtrar localmente un subconjunto diferente del que recibe la API.

11.6 Timeline básica

- Rango derivado de eventos y `published_at`; sin fechas futuras inventadas.
- Control: año, anterior, siguiente, play/pause.
- Play incrementa un año cada 600 ms; play se detiene al llegar al máximo o si hay reduced motion.
- Semántica: "Contexto documentado hasta YEAR"; no interpola estados ni resultados.
- Seleccionar un año actualiza mapa, lista, clusters y drawer.

11.7 Breadcrumb, escalas y navegación

- Escala: `WORLD`, `CONTINENT`, `COUNTRY`, `REGION`, `CITY`, `LOCAL_AREA`.
- Cada control calcula la extensión de las locations disponibles y ejecuta `flyTo`; los controles sin datos se deshabilitan.
- Breadcrumb se deriva de `parent_id`; click en un ancestro reencuadra la cámara.
- Doble click y botón `Volver` restauran la posición previa; no se restaura con animación de rotación automática.
- Viña del Mar es el centro inicial; el extent inicial incluye Viña, Valparaíso y contexto sudamericano.

11.8 Detalle y fuentes

- Drawer: nombre, ubicación, tipo, estado, evidencia, sector, año, resumen, impacto, actores, tags, eventos, status history, fuentes y relaciones.
- Cada fuente muestra nombre, tipo, fecha, confidence, snippet, URL con `target="_blank" rel="noopener noreferrer"` y última verificación.
- Empty state de relaciones: `No documented relations yet`.
- Escape cierra el drawer; al abrir, foco al heading; al cerrar, foco vuelve al elemento activador.
- Fuentes con URL no segura se muestran como texto no enlazado y se registra diagnóstico.

11.9 Stats

Strip con total de proyectos, locations, sources, fecha de última verificación y distribución por evidencia. Se usan etiquetas explícitas; no se muestran rankings, "leaders" ni métricas derivadas de densidad.

11.10 Render seguro

- Prohibido `innerHTML`, `insertAdjacentHTML` y `document.write` en `apps/web/src`.
- Todo dato externo se inserta con `textContent` o nodos DOM.
- URLs validadas con `URL`; sólo `https:` en producción.
- Mapbox y SVG se consideran límites de confianza y no reciben HTML remoto.

12. Seguridad e infraestructura

- Helmet, CORS allowlist, límite de body, request IDs y errores uniformes.
 - Sin secretos en el bundle; `VITE_MAPBOX_TOKEN` es público por diseño y debe restringirse por URL en producción.
 - `DATABASE_URL` obligatoria fuera de desarrollo; credenciales de Docker mediante `.env`.
 - `docker-compose.yml` publica DB sólo en `127.0.0.1:5432`, agrega `pg_isready` healthcheck y elimina `container_name` fijo.
 - API y web tienen Dockerfiles multi-stage no-root; `docker-compose.prod.yml` queda preparado para Dokploy pero no despliega en esta entrega.
 - Token Mapbox ausente o inválido nunca bloquea la exploración de datos.
-

13. Verificación

13.1 Unitarias

- `packages/contracts`: vocabularios, schemas y rechazo de valores inválidos.
- Normalizadores de API, selectores, filtros de descendientes, saneo de `source_url` (sólo `http` / `https`).
- `renderMarkerSvg` y cluster count.
- Validadores de env y migraciones.

13.2 API

- Vitest + Supertest con repositorio fake para health, filtros, paginación, detail, errores y ausencia de POST.
- Tests de integración con `TEST_DATABASE_URL` y PostGIS en CI.
- Tests específicos: CORS allowlist, límite de pageSize, q con acentos, `locationId` descendiente, URL `javascript:` rechazada, mensaje 500 sin leak.

13.3 Frontend y E2E

- Playwright con fixtures deterministas y mock de API para el flujo principal.
- Casos: carga sin pantalla blanca, idle sin rotación, breadcrumb, seis escalas, búsqueda, filtros, clusters, drawer, timeline, fallback sin token, móvil 390×844, teclado.
- Axe: cero violaciones críticas en home, fallback y drawer.
- `prefers-reduced-motion: reduce`: ausencia de animaciones de pulso/órbita.

13.4 Gates

```
npm run lint
npm run typecheck
npm test
npm run build
npm run check
```

No se acepta una entrega con un gate omitido. Los cuatro gates (`lint`, `typecheck`, `test`, `build`) corren siempre. Sólo los tests de integración contra PostgreSQL se omiten localmente cuando `TEST_DATABASE_URL` no está definida, y el runner de tests lo reporta explícitamente en la salida (`skipped: db integration`), nunca como `passed`. En CI la variable siempre está definida y esos tests se ejecutan.

14. CI y entrega

`.github/workflows/ci.yml`:

1. Node 22 LTS.
2. `npm ci`.
3. `npm run lint`, `npm run typecheck`, `npm test`, `npm run build`.
4. Job PostGIS: servicio `postgis/postgis:16-3.4`, `TEST_DATABASE_URL`, migraciones, seed e integración.
5. Artefactos de cobertura y reporte Playwright.

Commits propuestos, sin amend ni force push:

1. `chore: initialize repository and workspace tooling`
2. `docs: add core navigation design specification`
3. `feat(api): add contracts migrations repositories and endpoints`
4. `feat(web): add modular atlas shell map and fallback`
5. `feat(web): add filters timeline detail and hierarchy navigation`
6. `test: add unit integration e2e and accessibility coverage`
7. `chore: add docker ci and operational documentation`
8. `docs: finalize audited seed and delivery guide`

Push a `origin/main` sólo después de:

- todos los checks verdes;
- revisión de `git status`, diff y archivos sensibles;
- confirmar que no hay `.env`, `node_modules`, `dist` ni secretos;
- verificar credenciales de GitHub.

15. Criterios de aceptación

1. `npm run check` pasa en local sin Docker y en CI.
2. La web carga con respuesta 200 de la API y nunca queda en blanco.
3. El globo permanece quieto durante 3 segundos de inactividad.
4. El mapa ocupa al menos 70% del área visible en desktop de 1440×900, medido sobre el área del viewport sin contar el header y el strip inferior.
5. Los pins no son círculos sólidos; cada nivel usa su forma jerárquica.
6. WORLD, CONTINENT, COUNTRY, REGION, CITY y LOCAL_AREA cambian cámara y breadcrumb.
7. Búsqueda y filtros actualizan mapa, clusters, lista y `matchingCount` de forma consistente; los `totals` globales permanecen visibles y rotulados como globales.
8. Los conteos de clusters reflejan los filtros activos.
9. El drawer muestra estado, evidencia, freshness, fuente y snippet; funciona en móvil.
10. La timeline filtra por años documentados y no inventa estados futuros.
11. Sin token o con token inválido aparece fallback con diagnóstico y los datos siguen navegables.
12. `POST /api/projects` no existe como ruta de escritura.
13. CORS rechaza Origins no permitidos.
14. No hay `innerHTML` con datos externos y `javascript:` se rechaza.
15. El seed contiene 5 proyectos, 11 locations, al menos 5 fuentes primarias, al menos 5 events y 0 relations inventadas.
16. La integración confirma paridad entre fixtures y filas de DB.
17. Reduced motion elimina pulsos y órbitas.
18. El flujo principal se puede completar sólo con teclado.
19. Axe no reporta violaciones críticas en las vistas principales.

20. `docker compose up -d` produce DB saludable y `npm run db:setup` aplica migraciones.
21. README documenta instalación, token, migraciones, arquitectura, API y auditoría de fuentes.
22. El repositorio se inicializa, conserva commits por fase y se sube a [LeonardoPS1/ia-world-atlas](#).

16. Riesgos y mitigaciones

Riesgo	Mitigación
Mapbox sin token, cuota o WebGL2	Fallback con diagnóstico, eventos registrados antes de <code>style.load</code> , timeout de estilo
Datos de la conversación sin evidencia	Seed auditado contra fuentes primarias; claims excluidos explícitamente
Cambios en volúmenes PostGIS existentes	Migración <code>002_upgrade_v4.sql</code> y <code>db:reset</code> documentado
Clusters con filtros inconsistentes	Filtro server-side como fuente única; selector único y test específico
E2E flaky con mapa real	Fixtures deterministas, waits explícitos y assertions de estado, no de pixels
Falsa sensación de “verificado”	<code>VERIFIED</code> reservado a auditoría; stats con etiquetas de evidencia
Scope creep hacia producto completo	No objetivos explícitos y backlog separado para Fase 2
Push sin autenticación	Verificación previa de remote y credenciales; detener sin tocar el repo si falla

17. Backlog posterior

- Ingesta, normalización, deduplicación y entity resolution.
- Admin/backoffice protegido, aprobación, merge y status history UI.
- AI Pulse, grafo visual, Story Mode, Future Radar, AI Stack, AI Invisible y Possibility Lab.
- Comparador, My Atlas, follow, notificaciones y autenticación.
- Vector tiles, carga por viewport, light cartográfico e i18n.

18. Registro de decisiones

- La primera entrega se centra en navegación, evidencia y confianza, no en news ingestion.
 - El mapa es la superficie principal; paneles son contextuales.
 - No hay rotación automática ni rankings.
 - La API es read-only en V1.
 - La fuente primaria, la fecha y la evidencia acompañan cada registro.
 - La estructura está orientada a Docker y Dokploy, pero el despliegue queda fuera de esta entrega.
-

19. Preguntas abiertas

No hay preguntas bloqueantes. Las decisiones de Fase 2 —proveedor de tiles, modelo de cuentas, automatismos de ingesta y monetización— no afectan el Núcleo Navegable.